

Docker Lab Report (Lab 2)

Name: _Tang Wenting__

ID: _202383910018__

Date: _2026.4.2__

1、Objective

This experiment aims to learn the basic concepts of Docker, including container execution, port mapping, data persistence, and multi-container management.

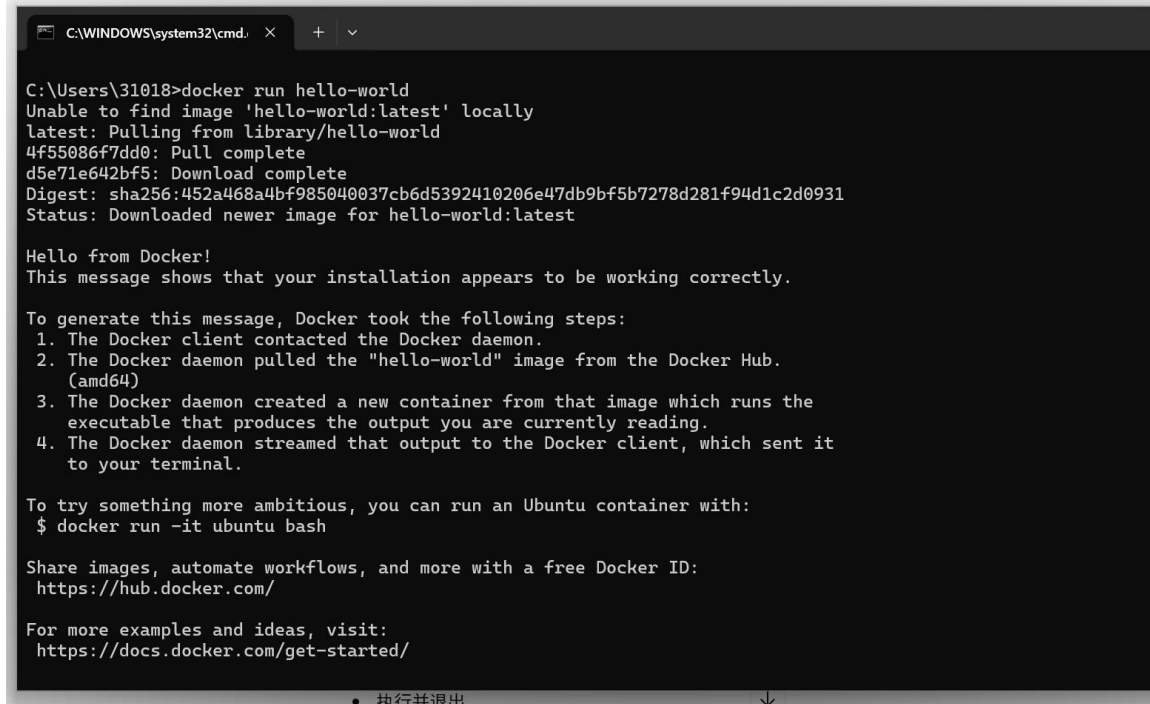
2、Contents

1. Run Docker containers
2. Publish container ports
3. Override container defaults
4. Persist container data
5. Share local files
6. Deploy multi-container applications

3、Steps

Step 1: Run a basic container

Command: `docker run hello-world`

A screenshot of a Windows command prompt window. The title bar shows the path 'C:\WINDOWS\system32\cmd...' and standard window controls. The command prompt shows the user running 'docker run hello-world'. The output indicates that the 'hello-world:latest' image was not found locally, so it was pulled from the Docker Hub library. It shows the pull progress, completion, and the image digest. The status is 'Downloaded newer image for hello-world:latest'. The container then runs and outputs 'Hello from Docker!' followed by a message stating that the installation appears to be working correctly. It then lists the steps Docker took to generate this message: 1. The Docker client contacted the Docker daemon. 2. The Docker daemon pulled the 'hello-world' image from the Docker Hub. (amd64) 3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading. 4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal. Finally, it suggests trying something more ambitious like running an Ubuntu container with the command '\$ docker run -it ubuntu bash' and provides links to Docker Hub and Docker documentation for more examples and ideas.

```
C:\Users\31018>docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:452a468a4bf985040037cb6d5392410206e47db9bf5b7278d281f94d1c2d0931
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

This step is used to verify whether Docker is installed and functioning correctly. When executing this command, Docker first checks if the image exists locally. If not, it automatically pulls the image from Docker Hub. Then, Docker creates a container, runs the program inside it, and outputs the result to the terminal.

Figure 1 shows the successful execution of the hello-world container, indicating that Docker is properly installed and working.

Step 2: Run an nginx container with port mapping

Command: `docker run -d -p 8080:80 nginx`

In this step, an nginx container is started in detached mode. Port mapping is configured so that port 8080 on the host machine is mapped to port 80 inside the container. This allows users to access the web service through a browser using `http://localhost:8080`.

```
C:\Users\31018>docker run -d -p 8080:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
5e815e07e569: Pull complete
cde7a05ae428: Pull complete
ec781dee3f47: Pull complete
3189680c601f: Pull complete
bb3d0aa29654: Pull complete
587e3d84dbb5: Pull complete
510ddf6557d6: Pull complete
669e0ab8e7fa: Download complete
96a6cfe061e0: Download complete
Digest: sha256:7150b3a39203cb5bee612ff4a9d18774f8c7caf6399d6e8985e97e28eb751c18
Status: Downloaded newer image for nginx:latest
eb2c7cea6ac7d87ab25fe9105d885934c417ec84f6c17ac009756026d78e888b
```



Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

Step 3: Run a container with interactive mode

Command: `docker run -it ubuntu bash`

Enter the container

```
C:\Users\31018>docker run -it ubuntu bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
817807f3c64e: Pull complete
e4a1e8de092c: Download complete
Digest: sha256:186072bba1b2f436cbb91ef2567abca677337cfc786c86e107d25b7072feef0c
Status: Downloaded newer image for ubuntu:latest
root@ac4c24bb167a:/# echo "Hello Docker"
Hello Docker
root@ac4c24bb167a:/#
```

This step demonstrates how to enter a container interactively. The `-it` option allows the user to access the container's terminal and interact with it directly. Inside the container, Linux commands can be executed, which helps understand the container environment.

Step 4: Persist container data using volumes

Command: `docker run -d -p 8081:80 -v mydata:/usr/share/nginx/html nginx`

```
C:\Users\31018>docker run -d -p 8081:80 -v mydata:/usr/share/nginx/html nginx
0804453f0fd5c4320e4d2e04f86907dadfb62b9dc3bf1535f58b10900b2b1962
```

This step demonstrates data persistence using Docker volumes. A volume named `mydata` is created and mounted to the nginx default web directory. This ensures that data stored in the container remains available even if the container is removed.

Step 5: Share local files with the container

```
C:\Users\31018>docker run -d -p 8082:80 -v C:\Users\31018\Downloads\index1.html nginx
81116d823a31f7ebcf375c272f17258955cbfe473dd961fa41e5c900fbd43dad
```

This step aims to mount a local file or directory into the container so that nginx serves custom HTML content.

During this step, an issue was encountered where the webpage did not display the custom HTML file. This happened because the volume mount command was incorrect. A file path was provided without specifying the correct target path inside the container, and the filename mapping was missing. As a result, nginx continued to serve its default page. So, I need to change.

```
C:\Users\31018>docker run -d -p 8082:80 -v C:\Users\31018\Downloads\index1.html:/usr/share/nginx/html/index.html nginx
0f910f6873351bfb730f756b560de0d573c53bee1e80a34d0656e29c0cdf33b2
docker: Error response from daemon: failed to set up container networking: driver failed programming external connecti
ty on endpoint objective_rubin (38580dbcecab337ffc53ffc5f7d5235e2fcce0dd7eb2cc9fa94e35a654da8f57): Bind for 0.0.0.0:8082
failed: port is already allocated

Run 'docker run --help' for more information
```

Another issue occurred when port 8082 was already in use by another running container. This prevented the new container from starting. To resolve this, the port was changed to 8083.

```
C:\Users\31018>docker run -d -p 8083:80 -v C:\Users\31018\Downloads\index1.html:/usr/share/nginx/html/index.html nginx
cdf78366a4711f593d1e591e6df86cb6a26b7e83cf8cb290f7b99ceb47956a89
```

After fixing the port issue, the container started successfully, and nginx was running correctly. However, a "404 Not Found" error appeared because the browser requested `login.html`, while the actual mounted file inside the container was `index.html`. Since nginx could not find `login.html`, it returned a 404 error. The 404 Not Found error occurred because the browser requested `login.html`, while the actual mounted file inside the container was `index.html`, so nginx could not find the requested page.

After correcting the file path and access URL, the custom page was successfully displayed.

```
C:\Users\31018>docker run -d -p 8084:80 -v C:\Users\31018\Downloads\index1.html:/usr/share/nginx/html/login.html nginx
6579f2fd035edaf6f84ff02591b13c9d5f9fd3f0df247ac9751f737b29b81967
```



Note: Instead of removing the existing containers, new containers were created. As a result, multiple containers were running simultaneously, but they were started independently using docker run.

☰ ☐ Only show running containers

☐		Name	Container ID	Image	Port(s)	CPU (%)	Las	Actions
☐	●	cranky_bohr	eb2c7cea6ac7	nginx	8080:80 ↗	0%	30 r	⚙️ ▶️ ⋮ 🗑️
☐	○	relaxed_lamarr	ac4c24bb167a	ubuntu		0%	29 r	⚙️ ▶️ ⋮ 🗑️
☐	●	pensive_samm	0804453f0fd5	nginx	8081:80 ↗	0%	26 r	⚙️ ▶️ ⋮ 🗑️
☐	●	jolly_wilbur	81116d823a31	nginx	8082:80 ↗	0%	20 r	⚙️ ▶️ ⋮ 🗑️
☐	○	objective_rubin	0f910f687335	nginx	8082:80	0%		⚙️ ▶️ ⋮ 🗑️
☐	●	flamboyant_jen	cdf78366a471	nginx	8083:80 ↗	0%	15 r	⚙️ ▶️ ⋮ 🗑️
☐	○	magical_samm	b1e8ea5805f0	nginx	8083:80	0%		⚙️ ▶️ ⋮ 🗑️
☐	●	nifty_galileo	6579f2fd035e	nginx	8084:80 ↗	0%	4 m	⚙️ ▶️ ⋮ 🗑️

Showing 10 items

Step 6: Deploy a multi-container application using Docker Compose

First, a docker-compose.yml file was created to define multiple services (nginx and redis).

```

version: '3'
services:
  web:
    image: nginx
    ports:
      - "8085:80"
  redis:
    image: redis

```

Then the following command was executed: docker compose up

```
C:\Users\31018>docker compose up
no configuration file provided: not found
```

```
C:\Users\31018>
```

Initially, an error occurred because Docker Compose could not find the configuration file. This happened because Docker only searches for the docker-compose.yml file in the current directory. After navigating to the correct directory, the command was executed successfully.

```
C:\Users\31018>cd Downloads
C:\Users\31018\Downloads>docker compose up
time="2026-04-02T11:07:50+08:00" level=warning msg="C:\\Users\\31018\\Downloads\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 12/12
✔Image redis Pulled 19.3s
✔Network downloads_default Created 0.1s
✔Container downloads-web-1 Created 0.2s
✔Container downloads-redis-1 Created 0.2s
Attaching to redis-1, web-1
web-1 | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration

redis-1 | Starting Redis Server
web-1 | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/

web-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
redis-1 | 1:C 02 Apr 2026 03:08:11.399 * o00o000o000o Redis is starting o00o000o000o
redis-1 | 1:C 02 Apr 2026 03:08:11.399 * Redis version=8.6.2, bits=64, commit=00000000, modified=1, pid=1, just started

web-1 | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
redis-1 | 1:C 02 Apr 2026 03:08:11.399 * Configuration loaded
web-1 | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
web-1 | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
web-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
```

```
so
redis-1 | 1:M 02 Apr 2026 03:08:11.415 * <ReJSON> Created new data type 'ReJSON-RL'
redis-1 | 1:M 02 Apr 2026 03:08:11.416 * <ReJSON> version: 80600 git sha: unknown branch: unknown
redis-1 | 1:M 02 Apr 2026 03:08:11.416 * <ReJSON> Exported RedisJSON_V1 API
redis-1 | 1:M 02 Apr 2026 03:08:11.416 * <ReJSON> Exported RedisJSON_V2 API
redis-1 | 1:M 02 Apr 2026 03:08:11.416 * <ReJSON> Exported RedisJSON_V3 API
redis-1 | 1:M 02 Apr 2026 03:08:11.416 * <ReJSON> Exported RedisJSON_V4 API
redis-1 | 1:M 02 Apr 2026 03:08:11.416 * <ReJSON> Exported RedisJSON_V5 API
redis-1 | 1:M 02 Apr 2026 03:08:11.416 * <ReJSON> Exported RedisJSON_V6 API
redis-1 | 1:M 02 Apr 2026 03:08:11.416 * <ReJSON> Enabled diskless replication
redis-1 | 1:M 02 Apr 2026 03:08:11.416 * <ReJSON> Initialized shared string cache, thread safe: true.
redis-1 | 1:M 02 Apr 2026 03:08:11.416 * Module 'ReJSON' loaded from /usr/local/lib/redis/modules//rejson.so
redis-1 | 1:M 02 Apr 2026 03:08:11.416 * <search> Acquired RedisJSON_V6 API
redis-1 | 1:M 02 Apr 2026 03:08:11.418 * Server initialized
redis-1 | 1:M 02 Apr 2026 03:08:11.418 * Ready to accept connections tcp
redis-1 | 1:M 02 Apr 2026 03:08:11.418 # WARNING: Redis does not require authentication and is not protected by network restrictions. Redis will accept connections from any IP address on any network interface.

View in Docker Desktop View Config Enable Watch Detach
```

The log message: redis-1 | Ready to accept connections tcp

indicates that the Redis container has started successfully and is ready to accept connections. This confirms that multiple containers (nginx and redis) are running together as part of a multi-container application.

4、 Conclusion and reflection

Through this experiment, I gained a comprehensive understanding of Docker and its practical applications. I learned how to create and run containers, configure port mapping, and implement data persistence using volumes. In addition, I understood how to share local files with containers and how Docker enables lightweight and efficient application deployment compared to traditional virtual machines.

During the experiment, I encountered several common issues, such as incorrect volume mounting paths and port conflicts. By troubleshooting these problems, I developed a deeper understanding of how Docker manages file systems and networking. For example, I realized that an incorrect mapping path would cause the container to use default content, and that port conflicts prevent containers from starting properly.

Furthermore, I learned how to use Docker Compose to deploy and manage multiple containers simultaneously. This approach simplifies the process of running multi-service applications and improves overall system organization and scalability.

Overall, this experiment not only strengthened my practical skills in using Docker, but also enhanced my ability to analyze and solve problems in containerized environments. These skills are essential for modern software development, cloud computing, and DevOps practices.